

WordPress Theme Conversion

Master Prompt

HTML to WordPress Parent/Child Theme with ACF Free
19 Non-Negotiable Rules • 2-Phase Process • Content Audit System
Paste this entire prompt at the start of any new conversion project.

CRITICAL GOAL

First preserve the website exactly as it already exists. That means:

- Keep all pages, sections, text, images, and links
- Keep all page order and section order initially
- Keep the same HTML-based design direction
- Keep spacing, typography hierarchy, layout structure, button positions, and visual styling as close as possible to the original HTML version

Do NOT redesign, rewrite content, simplify sections, merge sections, delete sections, invent new content, change wording, or improve the design unless required for exact theme implementation. The WordPress theme must look and feel like the original HTML-created pages first.

WORK IN 2 PHASES

Phase 1 — Exact HTML to WordPress Conversion

Rebuild the existing HTML site into WordPress theme files so the frontend output matches the original HTML version as closely as possible. For every page:

- Reproduce the same structure, section layout, and content placement
- Reproduce the same class structure wherever possible
- Reproduce the same headings, text, images, and buttons exactly
- Reproduce the same visual hierarchy exactly

Important:

Before applying any architectural improvements, first ensure the WordPress version mirrors the original HTML pages.

Phase 2 — Apply the 19 Theme Rules

After the exact design/content match is achieved, refactor the theme to follow the rules below WITHOUT changing the frontend appearance or page content. The final output must keep the same design as the original HTML site while following all 19 rules.

NON-NEGOTIABLE CONSTRAINTS

- 1. Content must remain exactly the same as the original HTML pages
- 2. Frontend design must remain the same

- 3. HTML structure may be cleaned for WordPress/semantic reasons, but visual output must remain the same
- 4. If a rule conflicts with the original implementation, solve it in backend/theme architecture, not by changing design
- 5. Do not use page builders
- 6. Do not use ACF Pro-only features (no repeater, flexible content, options pages)
- 7. Do not hardcode content in templates if it should be manageable by ACF or defaults.php
- 8. Any empty field should output nothing
- 9. Keep the system scalable and developer-friendly

IMPLEMENT THESE 19 RULES EXACTLY

Rule 1 — Child Theme Architecture

- Create a minimal custom parent theme with only: style.css (metadata), index.php (fallback), functions.php (basic supports)
- Parent theme must contain no styling, no layout logic, no third-party dependency
- Build the full site in the child theme only
- Child theme style.css must declare Template: parent-theme-folder-name
- Child theme functions.php uses constants: THEME_DIR, THEME_URI, THEME_VERSION
- All includes use require_once THEME_DIR . '/path/file.php'
- Output installable .zip files for both parent and child themes

Rule 2 — ACF Fields for Every Editable Element

MANDATORY: Complete the Content Audit Table BEFORE writing any ACF code. Do NOT write any ACF registration code until the audit table is complete and approved.

Step A: Content Audit Table

For every page, for every section, create a table listing every visible element:

#	Page	Section	Element	Type	ACF Field Name
1	Home	Hero	Title	text	home_hero_title
2	Home	Hero	Description	textarea	home_hero_desc
3	Home	Hero	Button text	text	home_hero_btn_text
4	Home	Hero	Button URL	url	home_hero_btn_url
5	Home	Values	Card 1 title	text	home_values_card_1_title
6	Home	Values	Card 1 text	textarea	home_values_card_1_text
7	Home	Values	Card 1 icon	text	home_values_card_1_icon
8	Home	Values	Card 2 title	text	home_values_card_2_title
...	...	...	...	...	...

Count the items: If a section shows 4 cards with 3 properties each, the table must show 12 rows — not 3

Loop items: If a section shows 6 timeline rows with 2 columns each, the table must show 12 rows — not 2

Cross-check: Every array item in defaults.php must have a matching ACF field

Step B: Registration

- Register via `acf_add_local_field_group()` in dedicated PHP files (one per page, inside `inc/acf/`)
- Hook to `acf/init` action
- Wrap in: `if ( ! function_exists( 'acf_add_local_field_group' ) ) return;`
- Use ACF Free only
- Allowed types: text, textarea, image (return URL), true_false, number, url, email, select, tab, message
- No repeater, no flexible content, no options page

Step C: Repeating Content (ACF Free Workaround)

ACF Free has no repeater field — use individual numbered fields instead:

```
card_1_title, card_1_text, card_1_image
card_2_title, card_2_text, card_2_image
card_3_title, card_3_text, card_3_image
```

- Field name format: `{page}_{section}_{item}_{n}_{property}`
- Field key format: `field_{page}_{section}_{item}_{n}_{property}` (globally unique)
- `defaults.php` stores the SAME content as PHP arrays — this is the FALLBACK, not the only place to edit
- Template reads ACF first, falls back to defaults: `get_field('card_' . $n . '_title') ? : $default['title']`

The client must be able to edit EVERY visible text, image, and link from wp-admin without touching code.

Step D: Location Rules

- Page templates: `'param' => 'page_template', 'value' => 'page-{slug}.php'`
- Homepage: `'param' => 'page_type', 'value' => 'front_page'`
- Organize fields with ACF tab type — one tab per section

Step E: Verification

- Every visible text string in the HTML has a matching ACF field
- Every image src has a matching ACF image field
- Every link href + text has two ACF fields (url + text)
- Every repeating group has $N \times$ fields (not just 1 set of section-level fields)
- Total field count matches: compare `defaults.php` entries vs ACF field count

Rule 3 — Empty Fields Output Nothing

- If a field is empty, output nothing — no placeholder text, no lorem ipsum
- Use a wrapper helper that checks ACF first, falls back to `defaults.php`, returns empty if both empty

- defaults.php contains ALL original HTML content as PHP arrays by page and section
- Template uses: `<?php if ( $value ) : ?>...<?php endif; ?>`
- Site works identically with zero ACF data entered (shows defaults)

Rule 4 — Show, Hide, and Reorder Sections

- Every section gets: {section}_show (true_false, default true) + {section}_order (number, defaults 10, 20, 30...)
- If show is false, do not render the section
- Build a reusable render function that collects, filters, sorts by order, and renders via `get_template_part()`
- defaults.php includes show and order for every section

Rule 5 — BEM CSS Classes on Every Element

- Use strict BEM naming: Block__Element--Modifier
- Blocks: .hero, .nav, .section, .footer, .card
- Elements: .hero__title, .card__text, .nav__link
- Modifiers: .btn--primary, .section--dark
- No utility classes (no .mt-4, .text-center) and no framework classes (no Bootstrap, Tailwind)

Rule 6 — CSS Variables for Colours

- Define all brand colours as CSS custom properties in :root
- Minimum: --primary-color, --secondary-color, --accent-color, --text-color, --bg-color, --border-color
- Include dark/light variants — zero hardcoded hex/rgb in component styles
- Colors editable via WordPress Customizer (Rule 19)

Rule 7 — Typography Variables with Fluid Scaling

- Font families as CSS custom properties
- H1–H3 use clamp(): --h1-size: clamp(2.25rem, 5vw, 4.5rem)
- H4+ and body can be fixed (rem or px)
- Typography editable via WordPress Customizer (Rule 19)

Rule 8 — Responsive Section Padding

- Section padding uses CSS variables and clamp()
- --section-padding: clamp(3rem, 6vw, 7rem)
- No fixed px padding on sections or containers
- Padding editable via WordPress Customizer (Rule 19)

Rule 9 — Inner Container and Content Spacing

- Define --container-max-width, --container-padding with clamp(), gap variables (sm, md, lg, xl)

- Center containers with auto margins
- Container editable via WordPress Customizer (Rule 19)

Rule 10 — Each Section is Self-Contained

- Each section is one PHP file in template-parts/sections/
- Each section file fetches its own data (ACF fields + defaults fallback)
- Each section outputs complete HTML (opening and closing tags)
- Sections can be moved between pages without breaking
- No global variables or shared state between sections

Rule 11 — Separate Responsive CSS File

- Desktop/base styles in main.css — media queries ONLY in responsive.css
- responsive.css contains ONLY @media blocks — zero standalone rules
- Load responsive.css after main.css via wp_enqueue_style
- Minimum breakpoints: 1024px (tablet), 768px (mobile)
- Recommended additional: 480px (small mobile), 1440px+ (large screens)
- Include prefers-reduced-motion: reduce and @media print styles

Rule 12 — Contact Form 7 for All Forms

- CF7 shortcode stored in an ACF text field
- Template renders with do_shortcode() — fallback message when empty
- CF7 form styled to match site design in main.css

Rule 13 — One H1 Per Page

- Only one H1 per page — the page hero/title section only
- All other section headings must be H2 — subsections use H3/H4
- Never skip heading levels (H1 → H2 → H3, never H1 → H3)

Rule 14 — Semantic HTML5 Structure

- header.php: <!DOCTYPE html>, <html>, <head>, <header>, <nav>, <main>
- footer.php: </main>, <footer>, </body>, </html>
- wp_head() in <head>, wp_footer() before </body>, wp_body_open() after <body>
- language_attributes() on <html>, body_class() on <body>

Rule 15 — Image Alt Text with 3-Tier Fallback

- Tier 1: WordPress Media Library alt text
- Tier 2: Dedicated ACF alt text field
- Tier 3: Empty string alt="" (valid HTML)
- All output uses esc_url() for src and esc_attr() for alt

Rule 16 — Safe HTML in ACF Textarea Fields

- Textarea output uses `wp_kses_post()` — allows `<br>`, `<strong>`, `<em>`, `<a>`, `<ul>`, `<li>`, `<p>`
- Plain text fields use `esc_html()` — URLs use `esc_url()` — attributes use `esc_attr()`
- Never use raw echo without escaping

Rule 17 — WordPress Security Basics

- Remove WordPress version meta: `remove_action( 'wp_head', 'wp_generator' )`
- Disable XML-RPC if not required: `add_filter( 'xmlrpc_enabled', '__return_false' )`
- Remove REST API user enumeration for non-logged-in users
- Add X-Content-Type-Options: nosniff header
- Never commit `.env`, credentials, or debug logs

Rule 18 — Performance: Lazy Loading & Font Strategy

- Hero/above-fold images: `loading="eager"` and `fetchpriority="high"`
- All below-fold images: `loading="lazy"`
- Google Fonts loaded with `font-display: swap`
- CSS in `<head>`, JS deferred or in footer
- Images use `width` and `height` attributes to prevent layout shift

Rule 19 — WordPress Customizer for Design Variables

- All CSS custom properties from Rules 6–9 editable via WordPress Customizer
- Single panel ("Theme Design") with sections: Colors, Typography, Spacing, Container
- All controls use live preview (transport: `postMessage`)
- Live preview JS uses `document.documentElement.style.setProperty()`
- Sanitization: `sanitize_hex_color` for colors, `wp_strip_all_tags` for text
- Output as inline `<style>` in `wp_head` — only non-default values
- Files: `inc/customizer.php` + `assets/js/customizer-preview.js`

FILE/FOLDER STRUCTURE

child-theme/	
style.css	
functions.php	
defaults.php	← all fallback content
header.php	
footer.php	
front-page.php	← homepage template
page-{slug}.php	← page templates
inc/	
customizer.php	← WordPress Customizer
helpers/	
field-helpers.php	← ACF + defaults bridge
image-helpers.php	← 3-tier alt text fallback
section-helpers.php	← show/hide/reorder engine
acf/	
{page}-fields.php	← one per page
template-parts/	
sections/	
{page}-{section}.php	← one per section
assets/	
css/	
main.css	← base styles + CSS vars
responsive.css	← media queries ONLY
js/	
main.js	← frontend JS
customizer-preview.js	← live preview JS

OUTPUT EXPECTATIONS

- 1. Show the proposed theme structure
 - 2. Create the Content Audit Table for every page (Rule 2 — MUST be done before ACF code)
 - 3. Create the minimal parent theme
 - 4. Create child theme core files (functions.php, defaults.php, helpers)
 - 5. Create all page templates and section partials
 - 6. Create all ACF field groups (matching the audit table exactly)
 - 7. Create CSS architecture (main.css + responsive.css)
 - 8. Create WordPress Customizer (Rule 19)
 - 9. Confirm how each original HTML page has been preserved
 - 10. Create installable zip-ready parent and child themes
-

FINAL VERIFICATION CHECKLIST

Before finishing, verify every item:

- Frontend still matches original HTML design exactly
- All original pages still exist with unchanged content
- All 19 rules are implemented
- No ACF Pro features used
- Content Audit Table was completed and every listed field exists in ACF code
- Every repeating group has individual numbered ACF fields
- Empty fields output nothing
- defaults.php fallback works (site looks correct with zero ACF data entered)
- Show/hide toggles work for every section
- Order numbers reorder sections correctly
- Only one H1 per page
- responsive.css is separate and contains only @media blocks
- Parent theme remains minimal (3 files only)
- Child theme contains all actual work
- WordPress Customizer loads with all panels and live preview works
- All images have alt attributes
- All output properly escaped (esc_html, esc_url, esc_attr, wp_kses_post)
- Security hardening applied (Rule 17)
- Lazy loading applied to below-fold images (Rule 18)
- No hardcoded content in templates — all from ACF or defaults.php

Zero redesign. Preserve the original HTML website exactly first, then refactor into WordPress while applying the 19 rules without changing the frontend.